

CS320 Lab: A Genetic Algorithm for the Travelling Salesperson Problem

Created by Prof. Craig Partridge

Lab #9

In this lab, you are tasked with writing the critical portions of a genetic algorithm to solve the traveling salesperson problem (TSP).

1 The Assignment

You will write a routine `ga_tsp(initial_population, distances, generations)` which is given an initial population of valid Hamiltonian paths (`initial_population`, which is a list of paths [tuples]), a dictionary of distances (`distances`), and the number of generations (`generations`) that your algorithm should run. After completing the requisite number of generations, `ga_tsp()` should return the best path (a tuple of cities) from the final generation.

Note that a generation is defined as a single iteration of the algorithm (parent selection, crossover, selection of the fittest population) using a steady population across generations (the same number of children as there are parents). If you use a variation of the above approach, such as creating half the number of children compared to the population size, you'll need to perform twice the number of iterations (generations).

A path is a tuple listing all "cities" in the path. It assumed the path is a Hamiltonian circuit that completes by taking an edge from the last city listed back to the first city listed. Do not list the first city twice (at the start and end of the tuple)!

We are providing several supporting routines and files:

- `util.py` contains three utility routines. `cost(path, distances)` will return the cost of a path. As a debugging aid, `cost()` checks that the path and distance dictionary are valid and will return a massive cost value if not. Note that `cost()` may not detect all invalid paths. `valid_path(path)` confirms the path is internally sound (no repetitions). `best_path(list_of_paths, distances)` returns the best (lowest cost) path in `list_of_paths`.
- `load_dist(filename)` will load a dictionary of distances from the file named `filename` and can be imported from `load_dist.py`. The format of `filename` is individual lines of the form "A, B, distance" where A and B are city names and distance is a positive integer distance.

A line causes the distance from A to B and B to A to be loaded into the dictionary. A line starting with `#` is a comment. Comment lines and blank lines are ignored. `load_dist()` also ensures that the distance from a node to itself is always zero. You can update the A to B (B to A) distance later in the file – so if you want to slightly change the topology (aka distances), you can just add a custom update to the end of the file. `load_dist()` returns a list of all the city names and a dictionary of distances. `load_dist()` returns `None, None` if the file did not load properly.

- `tsp.py` if run at the command line with a filename, so `% python3 tsp.py filename` will load distances from filename, compute an initial population, and call `ga_tsp()` to run 500 generations and report if `ga_tsp()` found a better result than was in the initial population.
- `init_pop.py` contains two routines to create an initial population. These routines are used in `tsp.py`. One version of the routine, `init_pop(list, dist)`, takes a list of cities and the distance matrix and generates a random set of initial paths of appropriate size. The other version, `init_lousy()`, intentionally creates an initial set of paths with high costs – which may be useful for debugging. (Instructions for using `init_lousy()` are in the file and in comments in `tsp.py`).
- `two_opt.py` contains the 2-opt algorithm described in class, designed to use the lab’s cost function. You are not required to use 2-opt in your implementation. It is provided solely for your convenience.
- We are providing two sample distance files: `DIST6-simple` is a simple 6 city world with a clear best path. `DIST23-basic` is a 23 city world divided up into 3 districts (clouds).

1.1 Requirements

Reminder that every assignment has a code portion, which is worth 60 points (40 for correctness, 20 for literate programming), and a reflection (worth 40 points).

1.1.1 Code Requirements

If `ga_tsp()` is given `None` for any argument it should raise `ValueError("Invalid_argument")`.

If generations is not positive, `ga_tsp()` should again raise `ValueError` with the same message.

You can assume `distances` and `initial_population` are valid if non-`None`.

We have no big-O performance requirements for this assignment. Rather we will run some tests designed to see if your genetic algorithm finds better results than the best path in the initial population (you can assume we’ll rig the initial population so a range of better solutions are feasible).

1.1.2 The Reflective Essay

For this reflection essay, we ask you to write it as if you were writing an essay to the course instructor, Ariana Mims, describing your understanding of the lab, your thought processes, and how those thought processes evolved. This could be viewed as a more “traditional” essay than the previous essays in the semester. Keep the essay to no more than 250 words.

When writing your reflection, consider the following questions:

- What strategies did you use while creating your implementation? How did those strategies evolve over time? What did you try that did not work?
- Why did you choose the implementation you did? Why were other strategies less attractive?
- What challenges did you have in implementation?
- How did you improve your code after you got it running? Why did you make those changes?

Be sure to use transition phrases and connect your ideas so that your audience (Ariana) understands your problem-solving process as a process that moves from one step to the next.

Note the rubric includes more information about how your reflections will be graded, so you should consult the rubric to make sure you are meeting the requirements.

1.2 Tips

You may want to use the sorted routine in ways you have not used it before. Specifically, sorted can take a list of additional arguments that may be useful.

To sort a list of paths using the cost function you set the key value in sorted. But sorted assumes the key function takes a single value, the item whose cost is to be assessed. To tell sorted that cost takes two arguments and assign the value of the second argument, you need to import partial from functools and then write something like:

```
sorted(population, key=partial(cost, distances=dist))
```

where dist is the dictionary of distances. There's an example in `init_pop.py`.

If you wish, you can also instruct sorted to sort in reverse order (so from high to low) by setting `reverse=True`. Again, there's an example in `init_pop.py`.